Counting, Carrying, and Circuits

Week 13 Tuesday — Boolean algebra, and how a computer adds and multiplies

COMSC 2043 ● Discrete Structures and Critical Thinking

November 10, 2026

75 minutes. Schedule change: Boolean algebra moved up to today; trees are out of this week; Thursday is Show & Tell. Arc of the day: we already know how to carry in decimal. Strip the digits away and the same carrying works in any base. Binary is the base with two symbols. Then a Boolean circuit can do that carrying. Spine of evidence for the day: Sources, Rules/Assumptions, Work, Check, One-Sentence Summary.

Today's destination

- Notice what **carrying** really is, using the decimal addition you already know.
- See that any set of symbols can be a number system (circles, stars, faces, fish).
- Add and multiply in **binary**, and see it is the same method as the one you learned.
- Meet Boolean algebra and build a **half adder** and a **full adder** from it.
- See why adding 16 bits fast needs a cleverer design: **ripple delay** versus **carry lookahead**.

Run of show: 0:00 warm-up and AI check (8). 0:08 decimal carrying (10). 0:18 octal and hex (10). 0:28 invented symbols (8). 0:36 binary add (8). 0:44 binary multiply (8). 0:52 Boolean algebra (8). 1:00 half and full adder (12). 1:12 ripple delay and lookahead (use as extension if time is short; it is also Thursday-safe) (6). Exit ticket (3).

Count on your fingers or on paper, out loud if you like:

$$7 + 1 \quad 8 + 1 \quad 9 + 1 \quad 99 + 1$$

- What did you run *out of* at $9 + 1$?
- Where did the extra digit come from?

Let them say it: we ran out of digits. There is no single symbol bigger than 9, so we write a zero in the ones place and put a one in a new column. Do not give the word “carry” yet; let a student produce it.

0 1 2 3 4 5 6 7 8 9 **10**

- We have **ten** symbols: 0 through 9. Not 10 symbols plus "10".
- At the end of the symbols, we **reset the ones place to 0** and **add 1 to the next place**.
- The "10" is two digits: a 1 in the tens place and a 0 in the ones place.

Hit the point: 10 is not a new symbol. It is a one and a zero, two digits. The first time we need a second column is exactly when the first column runs out. Odometer picture: the right wheel hits 9, rolls over to 0, and bumps the next wheel by one.

The same move at $99 + 1$

$$\begin{array}{r} 9\ 9 \\ +\quad 1 \\ \hline 1\ 0\ 0 \end{array}$$

- $9 + 1$: ones place resets to 0, carry 1.
- $9 + 1$ (the carry): tens place resets to 0, carry 1.
- Nothing left to add to, so the carry becomes a new digit.

Place values: ones $= 10^0$, tens $= 10^1$, hundreds $= 10^2$. **A new column appears every time the last one fills.**

Walk it slowly. The carry ripples left. This ripple is the reason a circuit that adds is built from a chain of identical pieces; hold that thought for the full adder.

Check: use the carrying rule on bigger sums

$$\begin{array}{r} 58 \\ +\ 27 \\ \hline 85 \end{array} \qquad \begin{array}{r} 999 \\ +\ \ 1 \\ \hline 1000 \end{array}$$

Say aloud which columns carried, and what the carry is worth.

$58 + 27 = 85$ ($8{+}7 = 15$: write 5, carry 1; $5{+}2{+}1 = 8$). $999 + 1 = 1000$ (three carries). Have two students narrate.



Verified: $7+1 = 10$ in octal (decimal 8). $77+1 = 100$ in octal (decimal 64). Example sum in octal: $27 + 15 = 44$ ($7+5 = 12$ decimal $= 14$ octal: write 4 carry 1; $2+1+1 = 4$). Let the class check the last one.

Digits 0 1 2 3 4 5 6 7 8 9 A B C D E F. We borrow letters because we need six more symbols.

8 9 A B C D E F **10** 11

- 9 + 7 = 10: sixteen, so a one and a zero. A + 5 = F.
- F + 1 = 10. FF + 1 = 100 (that number is two hundred fifty-six).
- Place values: 1, 16, 256, 4096, ...

Verified: hex 9+7 = 10 (decimal 16), hex A+5 = F, hex F+1 = 10, hex FF+1 = 100 (decimal 256). Example: 3B + 1C = 57 (B+C = 11+12 = 23 decimal = 17 hex: write 7, carry 1; 3+1+1 = 5). Why hex exists: one hex digit is exactly four binary digits, so it is shorthand for binary. Save that for the end of binary.

In base b there are b symbols, the biggest is $b-1$.

	biggest digit $+1$	biggest two digits $+1$
base 10	$9+1 = 10$	$99+1 = 100$
base 8	$7+1 = 10$	$77+1 = 100$
base 16	$F+1 = 10$	$FF+1 = 100$

"10" always means one group of b and none left over. Only the name of the number changes.

This is the generalization slide. If they have it, the invented-symbol slide is easy. Ask: what would this row look like for base 2? (1+1 = 10, 11+1 = 100.) Do not reveal yet, just ask and park it.



This is the abstraction slide Jeremy asked for: a number system is a set of symbols plus an ordering plus the carry rule. The symbols are arbitrary. Ask the class to name the symbol that comes after fish-star-fish... wait for them to count.

Counting 0 to 8 with circles, stars, faces and fish

decimal	base 4	symbols
0	0	
1	1	
2	2	
3	3	
4	10	
5	11	
6	12	
7	13	
8	20	

Fill this in live if you want: cover the right column and let the class write it. Rows 0 through 8 here, 9 through 16 on the next slide.

decimal	base 4	symbols
9	21	●▼
10	22	●●
11	23	●■
12	30	■○
13	31	■▼
14	32	■●
15	33	■■
16	100	▼○○

Row 16 is the ▼○○ moment: the same thing as $99 + 1 = 100$.

Decimal 16 is 100 in base 4 (one group of four fours). Same place-value story: ones, fours, sixteens.



Check: base-4 3+1 = 10 (decimal 4). Base-4 3+3 = 12 (decimal 6 = one four plus two). The lesson: the symbols were never the point. The rule about carrying was. This sets up: now use the smallest possible set of symbols.

Only 0 and 1. Biggest digit is 1.

n	binary
0	0
1	1
2	10
3	11
4	100
5	101
6	110
7	111
8	1000

n	binary
9	1001
10	1010
11	1011
12	1100
13	1101
14	1110
15	1111
16	10000

Place values: 1, 2, 4, 8, 16, . . .

The first column fills at 1. So 1 + 1 = 10 (that is two).

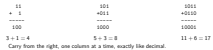
Pattern spotting: the ones column alternates 0 1 0 1; the twos column alternates in pairs; the fours in fours. Each column is half as fast as the one to its right. Verified table by Python. $8 = 1000$, $16 = 10000$.

Binary addition: the whole rule table

sum	result
$0 + 0$	0
$0 + 1$	1
$1 + 0$	1
$1 + 1$	10 (write 0, carry 1)

That is the entire addition table. With a carry in, the last case can reach $1 + 1 + 1 = 11$ (write 1, carry 1).

Compare to decimal, where the table has 100 entries (10 by 10). Binary has four. Everything else is the carrying procedure they already know.



Verified: 11+1 = 100 (3+1 = 4). 101+011 = 1000 (5+3 = 8). 1011+0110 = 10001 (11+6 = 17). Do the third one together, writing the carries above each column. Rightmost column 1+0 = 1; next 1+1 = 10 carry 1; next 0+1+1 = 10 carry 1; next 1+0+1 = 10 carry 1; the last carry becomes the new leftmost 1. Check by converting to decimal. Student tries: 0111 + 0101 (7+5 = 12 = 1100).

Check: your turn

Add in binary, then convert both sides back to decimal to check.

`0111 + 0101` `1111 + 0001`

One sentence: **where did each carry come from?**

$0111{+}0101 = 1100$ ($7{+}5 = 12$). $1111{+}0001 = 10000$ ($15{+}1 = 16$). The second one is the $99{+}1$ moment again: every column carries. Ask them to say the phrase “99 plus 1” back.

product	result
0×0	0
0×1	0
1×0	0
1×1	1

A product of single bits is **1 only if both are 1**. Hold that thought: it is AND.

Plant the connection now. Multiplication of bits = AND, and we will name it that in a few minutes.

Binary long multiplication: same as the method you know

Decimal: 23 × 14 = 322

```
   23
 x  14
 ----
   92   (23 x 4)
  230   (23 x 10)
 ----
  322
```

Binary: 5 × 3 = 15

```
    101
  x  11
  -----
    101   (101 x 1)
   1010   (101 x 1, shifted)
  -----
   1111
```

Each partial product is either the top number or zeros, then shifted. Then **add**, using the rule table.

Verified: 23 x 14 = 322; 101 x 11 = 1111 (5 x 3 = 15). Point out: in binary, each partial product is either a copy of the top number or all zeros, so there is no times table to memorize. Shifting left by one place is multiplying by two, the same way adding a zero on the right of 23 makes 230.

```
  11
x 10
----
  00
 110
----
 110
```

3 × 2 = 6

```
  110
x 101
-----
  110
 0000
11000
-----
11110
```

6 × 5 = 30

Verified: 11 x 10 = 110 (3 x 2 = 6). 110 x 101 = 11110 (6 x 5 = 30). Observe: multiplying by 10 (two) just shifts. Ask the class what multiplying by 100 does.

Boolean algebra has two values, true/false, which we write 1/0, and three basic operations:

AND ($A \cdot B$)

A	B	out
0	0	0
0	1	0
1	0	0
1	1	1

OR ($A + B$)

A	B	out
0	0	0
0	1	1
1	0	1
1	1	1

NOT ($\overline{A}$)

A	out
0	1
1	0

Compare AND with the one-bit multiplication table. They are identical. So single-bit multiplication is AND.



This is the critical-thinking beat of the day, and it is the usual Boolean-algebra stumble. The notation shares a plus sign. Ask: if I write $1 + 1 = 1$, is that wrong? It depends on the Rules/Assumptions line. Link to the course habit: define terms before judging a claim.

A fourth gate: XOR (exclusive or)

XOR is true when the inputs **differ**. Written $A \oplus B$.

A	B	XOR	binary $A + B$
0	0	0	00
0	1	1	01
1	0	1	01
1	1	0	10

Look at the last two columns: the **right-hand digit of the sum is the XOR**, and the **left-hand digit (the carry) is the AND**.

The key observation. The right digit of 0,1,1,0 is XOR's column. The left digit 0,0,0,1 is AND's column. So one bit of addition = one XOR gate for the sum bit and one AND gate for the carry bit. That is a half adder.

Adds two bits A and B. Outputs: sum $S = A \oplus B$, carry $C = A \cdot B$.



A	B	C	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

Read CS as a two-bit number: it equals $A + B$.

Every row of the table: A+B in binary is CS. $0+0 = 00$, $0+1 = 01$, $1+0 = 01$, $1+1 = 10$. The half adder cannot accept a carry coming in from the right, which is why it is called “half”. Rubric habit: truth table is the evidence.

The problem a half adder cannot solve

Inside a long sum, every column after the first receives three bits: A, B, and the **carry-in** from the column to its right.

```
  1111      <- carries
  01101
+ 00111
-------
  10100
```

That is three inputs (A, B, C_{in}) and two outputs (sum, carry-out). We need a **full adder**.

$13 + 7 = 20$. Column by column from the right: $1{+}1 = 10$, carry 1; $0{+}1{+}1 = 10$, carry 1; $1{+}1{+}1 = 11$, write 1, carry 1; $1{+}0{+}1 = 10$, carry 1; $0{+}0{+}1 = 1$. Point at any middle column: three bits arrive.

Sum $S = A \oplus B \oplus C_{in}$. Carry-out $C_{out} = A \cdot B + C_{in} \cdot (A \oplus B)$.

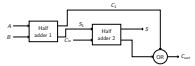
A	B	C_{in}	C_{out}	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

Read $C_{out}S$ as a two-bit number: it equals $A + B + C_{in}$ in every row.

Eight rows is every possible input. **Eight rows is a proof.**

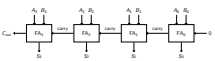
Verified by brute force: all eight rows satisfy 2*Cout + S = A + B + Cin. This is the Week 14 theme pulled forward: a truth table over every assignment is a complete check, one passing assignment is not.

Full adder = two half adders and an OR



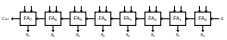
Half adder 1 adds $A + B$. **Half adder 2** adds that sum plus C_{in}. Either one can make a carry, so OR combines them.

If this diagram looks busy in the room, drop it and say it in words: first add A and B, then add the carry-in to that sum, and either step may produce a carry, so Cout = C1 OR C2. The two carries can never both be 1, which you can show with the truth table if time remains.



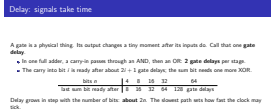
Each carry ripples left, one stage at a time: the same ripple as $99 + 1 = 100$.

Example: 0111 + 0101. FA0: 1+1 = 10, write 0 carry 1. FA1: 1+0+1 = 10, write 0 carry 1. FA2: 1+1+1 = 11, write 1 carry 1. FA3: 0+0+1 = 1. Result 1100 = 12. Overflow: if the answer needs more bits than we built, the carry-out is lost. 1111 + 0001 on a 4-bit adder gives 0000 with Cout = 1. Ask: so what is 4-bit 15 + 1? It wraps. Real computers use wider adders but the same idea. Multiplication in hardware is shifting plus these adders, with AND gates making the partial products.

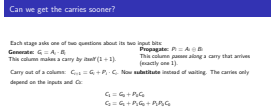


Worst case: 11111111 + 00000001 = 100000000 ($255 + 1 = 256$). The carry is created in the first column and must travel through **all eight**.

Verified: $255 + 1 = 256$ and the 8-bit adder returns 00000000 with carry-out 1. Same ripple as $99 + 1 = 100$, but now it is physical: each full adder cannot finish until the carry from the one to its right arrives. Ask: which column finishes last?



Model: every gate has delay 1; P = A xor B and G = A and B are ready at time 1; each carry takes an AND then an OR (2 more); the last sum bit is one XOR after its carry-in. That gives 2n for the last sum bit, which the checker script verifies for n = 4, 8, 16, 32, 64. The model is simplified (real gates differ in speed, wires add delay) but the trend is right: the delay is linear in n. Why it matters: a processor clock must wait for the slowest path to settle. If the adder takes 128 gate delays, the clock cannot tick faster than that. Critical-thinking beat: a model is useful and also leaves something out. Name what this one omits (wire delay, different gate speeds).

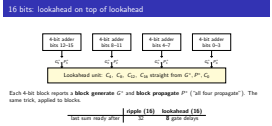


This is the key idea of carry look-ahead: stop waiting for the previous stage. Write C2 in terms of C1, then substitute C1. Every carry becomes a flat AND-OR of the P and G values, which are all ready at time 1. Interpret each term in words: C2 happens if column 1 generates one, or column 1 propagates a carry that column 0 generated, or column 1 and 0 both propagate the carry-in. Ask the class to say it in words.

$$C_1 = G_0 + P_0 C_0$$
$$C_2 = G_1 + P_1 G_0 + P_1 P_0 C_0$$
$$C_3 = G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_0$$
$$C_4 = G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 G_0 + P_3 P_2 P_1 P_0 C_0$$

Every carry is ready after **3 gate delays** (P and G: 1; the AND terms: 1; the OR: 1). Then sums: $S_i = P_i \oplus C_i$, **4 total**, against 8 for 4-bit ripple.

Counting: P, G at time 1; the AND of up to five signals at time 2; the OR at time 3; the sum XOR at time 4. That assumes gates with several inputs (up to five-input AND/OR). Real chips limit fan-in, which is why the next slide groups bits into blocks. Cost: many more gates than ripple. You trade hardware for speed.



Counting for the 16-bit design: P, G at 1; block P* at 2 (a four-input AND); block G* at 3; the block carries C4, C8, C12, C16 at 5 (two more levels); the carries inside each block at 7; sums at 8. Four times faster than ripple in this model. Verified: the checker script builds this exact design in code and tests it on 20,000 random 16-bit sums plus the edge cases 0xFFFF+1 and 0x00FF+1. Costs: more gates, wide gates, and more wires to route. Ask: when would you accept the extra hardware? (Fast general-purpose CPU) and when not? (Tiny cheap chip where speed does not matter.) Make the trade-off the takeaway, not the algebra.

Putting multiplication in the circuit

- Each partial product bit is an **AND** of one bit from each number.
- Shifting is just wiring each row one place to the left.
- The rows are added with **full adders**.

So a binary multiplier is AND gates plus the adders we just built.

Closing the loop on the multiplication slides. Do not draw the full array multiplier unless asked; the point is that nothing new was added. Everything was AND, XOR, OR. The connection to traditional multiplication: same partial products, same shifts, same final add.

An AI says: *"$A + B$ simplifies to A because it worked for $A = 1$, $B = 1$."*

- Which system is the plus sign in: Boolean OR or binary addition?
- Is one satisfying row a proof? What would be?

Write one sentence: what does a complete truth table prove that a single example cannot?

Target: one passing row is a witness for "there exists" and says nothing about "for all". A full truth table over every input is a proof for a finite domain. And the plus sign is ambiguous until the system is named. This foreshadows Week 14's equivalence checks.

Exit ticket (one sentence each):

1. In your own words: what does "carry" mean?
2. Which gate adds one bit, and which gate makes the carry?

Thursday: Show & Tell. **Next Tuesday:** finite-state machines (coin machine, and more) with Mermaid diagrams.

Answers: carry = when a place fills (reaches the base), reset it to 0 and add one to the next place. XOR gives the sum bit; AND gives the carry bit. Collect or ask for a show of fingers. Thursday is Show & Tell, as arranged.